BioSpecInfo

Architettura dell'Agente AI "Spectra" — BioSpecInfo

Samuele Pio Provenzano

Versione **bsi-v226** · commit `11a7ad7`

Generato il 2026-10-22

Documento generato automaticamente dal sorgente Markdown versionato nel repository. La versione e il commit stampati in ogni piè di pagina identificano esattamente lo stato del software descritto.

Architettura dell'Agente AI "Spectra" — BioSpecInfo

Campo	Valore
Progetto	BioSpecInfo — componente Spectra (copilota AI agentico)
Autore	Samuele Pio Provenzano
Relatore tesi	Prof. Savino Longo — Università degli Studi di Bari Aldo Moro
Componente	`bsi-ai-hub.js` — 6.254 righe, nessuna dipendenza runtime
Tipo	Agente conversazionale multi-provider con esecuzione di strumenti lato client
Repository	`samupropio1-ship-it/BioSpecInfo-v11`
Versione documentata	Service Worker `bsi-v226`

---

1. Sintesi esecutiva

Spectra è un **agente** — non un semplice wrapper su un modello linguistico. La differenza è misurabile: il modello non risponde a memoria sui dati quantitativi, ma **invoca strumenti deterministici** che calcolano il risultato, e il ciclo di controllo gli permette di **concatenare fino a 10 chiamate** verificando ogni passaggio prima di rispondere.

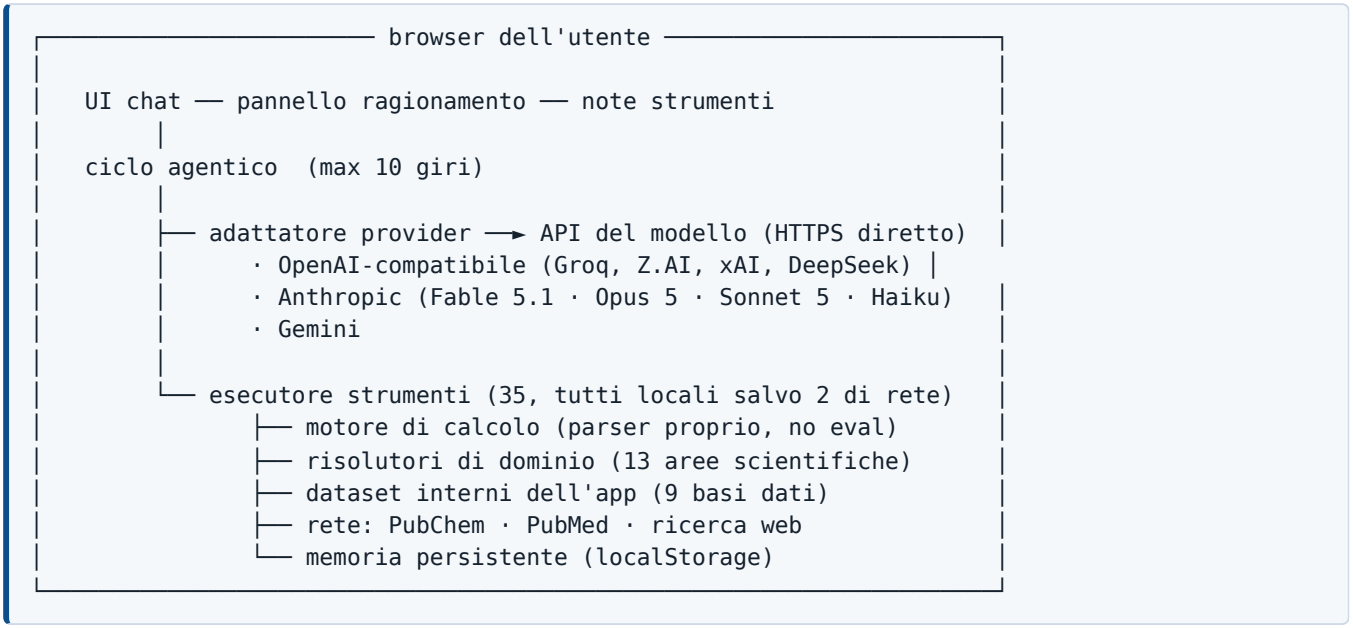
L'architettura affronta tre problemi che distinguono un agente utilizzabile da una demo:

1. **Fondatezza (*grounding*)** — un dato chimico inventato può essere pericoloso. 35 strumenti coprono calcolo, banche dati pubbliche e i dataset interni dell'applicazione; il prompt di sistema vieta esplicitamente di citare valori numerici a memoria.
1. **Trasparenza** — il ragionamento del modello è mostrato in tempo reale e conservato accanto alla risposta, insieme all'elenco degli strumenti effettivamente invocati. L'operato dell'agente è verificabile a posteriori.
1. **Portabilità fra fornitori** — un unico registro di strumenti viene tradotto nei tre formati di *function calling* oggi in uso (OpenAI-compatibile, Anthropic, Gemini), così l'agente funziona identico su dieci configurazioni di modello, quattro delle quali gratuite.

---

2. Architettura

2.1 Vista d'insieme



Non esiste un backend. Le chiavi API restano nel `localStorage` del browser e viaggiano solo verso il fornitore scelto: nessun dato transita da server di BioSpecInfo, che non esistono.

2.2 Ciclo agentico

Ad ogni giro il ciclo: invia la cronologia (ultimi 40 turni) più il prompt di sistema e lo schema dei 35 strumenti → riceve la risposta in *streaming* → se contiene chiamate a strumenti le esegue **tutte** → ricostruisce il turno assistente nel formato nativo del fornitore → ripete.

Tre condizioni di uscita: nessuna chiamata a strumenti (risposta finale), `MAX_ROUNDS = 10` esaurito, o interruzione dell'utente.

2.3 Registro strumenti neutrale rispetto al fornitore

Gli strumenti sono definiti una sola volta in JSON-Schema e tradotti a runtime:

Famiglia	Formato richiesto
Anthropic	`{name, description, input_schema}`
Gemini	`[{functionDeclarations: [...]}]`
OpenAI-compatibile	`{type:"function", function:{...}}`

Lo stesso vale per i messaggi: i turni con chiamate a strumenti vengono serializzati nella forma nativa di ciascun fornitore e conservati in un campo `_native`, così una conversazione resta coerente anche cambiando modello.

2.4 Nessun nome di modello cablato

Un nome di modello scritto nel codice è una bomba a orologeria: funziona finché il fornitore non ritira quel modello, e allora l'applicazione si ferma con un 404 che l'utente non può risolvere. È successo due volte in questo progetto:

Fornitore	Errore	Quando
Google	`models/gemini-1.5-flash is not found for API version v1beta`	ritiro da `v1beta`
Groq	`The model llama-3.3-70b-versatile does not exist or you do not have access to it`	deprecato il 17/06/2026

Il problema è strutturale su alcuni servizi: gli id dei modelli gratuiti cambiano di continuo, per costruzione. (È il motivo per cui OpenRouter è stato poi tolto dall'elenco, vedi 2.4.)

La correzione non è scrivere il nome nuovo — sarebbe la stessa bomba con la miccia più lunga — ma **togliere il nome** e chiederlo all'API, che sa quali modelli esistono *per quella chiave*. Il che risolve anche l'ambiguità del messaggio di Groq: «non esiste» **oppure** «non ci hai accesso» sono casi diversi, e un elenco per chiave li distingue.

Ogni configurazione con `modelliCandidati` parte da `model: null` e viene risolta a runtime. Le due famiglie hanno strategie diverse perché i loro nomi lo sono:

Gemini — punteggio per versione. I nomi seguono uno schema regolare (`gemini-<major>.<minor>-<famiglia>`), quindi si può ordinare: versione più recente, famiglia `flash`, con penalità per varianti sperimentali, `preview` e `-lite`; scartati i modelli senza `streamGenerateContent` e quelli non conversazionali (embedding, immagini, audio nativo, Live API). Quando uscirà Gemini 3.0 verrà scelto da solo.

Famiglia OpenAI — i candidati in ordine di preferenza. Qui i nomi non sono confrontabili fra loro (`openai/gpt-oss-120b` contro `qwen/qwen3.6-27b`), e un punteggio automatico sceglierebbe male. Si interroga `GET /models` e si prende **il primo candidato ancora esistente**: l'ordine della lista *è* la preferenza, e l'elenco serve a saltare quelli spariti. Solo se nessun candidato sopravvive si ricorre a un punteggio generico sui modelli disponibili — che scarta trascrizione, sintesi vocale, embedding e classificatori, e a parità d'altro preferisce il modello più grande.

Tre livelli di riserva in entrambi i casi: elenco dei modelli → sonda dei candidati → primo candidato, lasciando parlare l'errore vero della chiamata di generazione invece di inventarne uno.

La scelta resta in cache sette giorni per fornitore, legata a un'impronta FNV-1a a 32 bit della chiave (chiavi diverse vedono cataloghi diversi; la chiave in chiaro non viene mai duplicata). Se il modello viene ritirato *mentre* è in cache, il `404` — o un `400` il cui testo parla di modello, come fanno alcuni fornitori — invalida la cache, ririsolve e ritenta **una volta sola**: un flag impedisce il ciclo infinito quando anche il modello nuovo fallisce.

Anthropic resta l'eccezione voluta: i suoi modelli sono a pagamento, scelti esplicitamente dall'utente e con deprecazioni annunciate con largo anticipo.

I quattro servizi gratuiti, scelti uno per uno

La qualità gratuita non è tutta uguale, e la differenza conta: un modello da 8 miliardi di parametri non regge un problema di chimica fisica in più passaggi. L'elenco è stato **ridotto**, non allungato — due servizi sono stati tolti perché peggioravano la risposta invece di migliorarla.

Servizio	Modelli	Il ruolo
Groq	GPT-OSS 120B, Qwen3	Il cavallo da tiro: 131K di contesto, ~30 richieste/minuto
Google Gemini	Gemini Flash	Fino a 1M di contesto: documenti e PDF interi
Z.AI GLM	GLM-4.7-Flash	Gratuito **senza scadenza**: la riserva che resta quando i crediti finiscono

Tolti. *GitHub Models*: c'era, ed è stato tolto perché GitHub lo ha **ritirato del tutto il 30 luglio 2026** — playground, catalogo e API di inferenza spenti per tutti. Era stato aggiunto due giorni prima sulla base di pagine che descrivevano il servizio ancora attivo: la lezione è che verificare *che una cosa esista* non è verificare *che sia ancora viva*, e la ricerca da fare è «<nome> retired / shutdown / deprecated <anno>». Il guasto non si è fermato alla voce in tendina: il rango che gli era stato assegnato lo rendeva anche la scelta automatica della Modalità Nucleo.

Mistral: qualità media e piano gratuito che richiede il consenso all'uso dei dati per l'addestramento — un prezzo che non vale la pena pagare per materiale di tesi non pubblicato, quando esistono cinque alternative senza quella clausola. *OpenRouter*: modelli gratuiti piccoli e id che cambiano di continuo, inadatti a un agente che concatena dieci chiamate a strumenti. *Cerebras*: dall'agosto 2026 niente più piano senza carta, e contesto gratuito limitato a 8K — Spectra manda 32 definizioni di strumenti oltre alla cronologia, non ci sta.

Nessuno dei cinque eguaglia un modello di frontiera a pagamento sui problemi più difficili; Z.AI GLM ci va vicino, al prezzo di tetti di richieste bassi. Per questo la scelta resta dell'utente, invece di imporre un solo servizio.

La frontiera, a pagamento

I gratuiti bastano per studiare; per un problema difficile no. Il criterio di scelta è stato **GPQA Diamond** — domande di livello dottorato in fisica, chimica e biologia — perché è l'unico banco che misura ciò che quest'app chiede davvero.

Servizio	Perché c'è
GPT-5.6	94,6% su GPQA Diamond: il punteggio più alto oggi disponibile. ~4 $/M in ingresso
Gemini 3 Pro	Oltre 1M di contesto, e **la stessa chiave** del Gemini gratuito. ~2 $/M
DeepSeek V4	Ragionamento di fascia alta a ~0,66 $/M: il miglior rapporto qualità/prezzo
Grok 4.6	Primo su *tool calling* agentico e con il minor tasso di allucinazione — esattamente il profilo che serve a un agente che concatena strumenti. ~2 $/M

Servizio	Perché c'è
Claude (Fable 5.1, Opus 5, Sonnet 5)	Una sola chiave per tutti e tre; gli unici con il sandbox Python in Modalità Nucleo

Due configurazioni possono usare **la stessa chiave**: i quattro Claude sono un solo account Anthropic, e Gemini Flash e Gemini 3 Pro una sola chiave di AI Studio. `chiaveCondivisaCon` le fa puntare alla stessa voce, così l'utente la incolla una volta. Chi la cancella la cancella per tutti i gemelli — lasciarne una la farebbe riapparire sull'altro.

Il caso dei due Gemini ha richiesto di generalizzare il punteggio: condividono famiglia, endpoint e chiave ma vogliono modelli **opposti**. La preferenza è ora dichiarata in configurazione (`/flash/` contro `/pro/`) e vince sul resto del punteggio. Senza, la configurazione gratuita avrebbe scelto `gemini-3-pro`: una generazione più avanti, quindi con punteggio più alto, ma **a pagamento** — un modello che la chiave gratuita non può usare.

Nello stesso passaggio è emersa una seconda cosa: il numero minore nel nome è **opzionale**. Dalla generazione 3 Google lo ha tolto (`gemini-3-pro`, non `gemini-3.0-pro`), e la regex che pretendeva il punto valutava i modelli *più nuovi* come alias generici — quindi perdevano contro i vecchi, e l'aggiornamento automatico descritto in 2.4 non sarebbe mai scattato.

L'audit dei fornitori, e perché va rifatto

Il sistema si ripara da solo sui **nomi dei modelli** — interroga il catalogo e sceglie fra ciò che esiste. Non si ripara su un **servizio morto**: quello va verificato a mano, e il ritiro di GitHub Models ha mostrato cosa costa non farlo.

Verifica di settembre 2026, fornitore per fornitore:

Fornitore	Esito
Groq	Attivo. `llama-3.3-70b-versatile` **non è più servito** da agosto 2026 e `llama-3.1-8b-instant` è deprecato: tolti dai candidati, restano i sostituti indicati da Groq
Google Gemini	Attivo
NVIDIA NIM	Rimosso 09/2026: attivo, ma senza header CORS non è chiamabile da una pagina web
Z.AI	Attivo, GLM-4.7-Flash e 4.5-Flash ancora gratuiti
OpenAI, DeepSeek, Anthropic	Attivi
xAI	Attivo, ma i candidati erano di **due generazioni** precedenti: il vertice è `grok-4.6` dal 12/08/2026

Due trappole registrate perché non si ripetano. La prima: non mettere un modello a pagamento fra i candidati di una configurazione gratuita — vale per `glm-5.x` su Z.AI come valeva per `gemini-3-pro`, e la configurazione sceglierebbe qualcosa che la chiave gratuita non può usare. La seconda: un fornitore morto con un rango alto non rompe solo la voce in tendina, **diventa la scelta automatica** della Modalità Nucleo.

Quando un fornitore non risponde

Un `fetch` che fallisce **prima** di ricevere una risposta significa una di tre cose: rete assente, il servizio non accetta chiamate dirette dal browser (CORS), oppure il servizio non esiste più. Il browser, per ragioni di sicurezza, non permette di distinguere quale — l'errore è volutamente generico.

Prima questo fermava il turno. Ma se le tre cause non si distinguono, la risposta giusta è la stessa per tutte e tre: **passare a un altro fornitore** per cui l'utente ha una chiave, esattamente come per la quota esaurita. È il comportamento che avrebbe mascherato del tutto il ritiro di GitHub Models, invece di lasciare ogni richiesta bloccata su un servizio spento.

Restano fuori dalla riserva gli errori che si ripeterebbero identici ovunque — una chiave sbagliata, una richiesta malformata: provarli su tutti i fornitori per poi riportare l'ultimo errore a caso nasconderebbe la causa vera.

Togliere un servizio dall'elenco è un'operazione che rompe le cose, e merita una nota: chi lo aveva selezionato ha quel nome salvato in `localStorage`, e senza validazione si ritroverebbe `PROVIDERS[undefined]` — Spectra non si aprirebbe più. `getSavedProvider()` valida sempre contro il registro e ripiega su un servizio esistente.

2.5 Il proxy opzionale — Spectra senza chiave

La chiave API vive nel browser dell'utente: senza backend non c'e' alternativa, ed e' il limite dichiarato apertamente in fondo a questo documento. Ogni visitatore deve procurarsi la propria chiave gratuita e incollarla.

`proxy/spectra-proxy.js` e' la via d'uscita, e sta **fuori** dalla pagina: un Worker Cloudflare (piano gratuito) che tiene le chiavi come segreti lato server. Quando e' configurato, il browser non contiene nessuna chiave e chi apre BioSpecInfo usa Spectra senza inserire niente.

Non e' un semplice inoltro:

- **Il percorso passa tale e quale.** La rotta e' `/<fornitore>/<percorso>`, e

il resto viene inoltrato come arriva: il proxy non conosce i formati delle API e continua a funzionare quando Spectra cambia modello o parametri.

- **Piu' chiavi per fornitore, con subentro automatico.** Ogni segreto e' una

lista separata da virgole; su `429` (quota) o `401/403` (chiave revocata) si passa alla successiva nella stessa richiesta. Su `400` no: e' un errore nella richiesta e si ripeterebbe identico.

- **Lo streaming non viene bufferizzato**: il corpo della risposta passa

direttamente, altrimenti le risposte comparirebbero tutte insieme alla fine.

- **Non e' un relay aperto.** Credenziali che arrivano dal client

(`Authorization`, `x-api-key`, `?key=`) vengono scartate e sostituite: nessuno puo' usare il proxy per far viaggiare una chiave propria.

- **Tre freni** contro l'abuso: elenco di origini ammesse, limite per IP e

tetto giornaliero.

Lato Spectra l'innesto e' in un punto solo. `GET /stato` dice quali fornitori il proxy copre davvero, cosi' non viene offerto come "senza chiave" un modello per cui manca il segreto; `buildRequest` instrada al proxy **e omette del tutto l'autenticazione**; `chiaveDaUsare()` restituisce un segnaposto perche' i controlli "manca la chiave" non blocchino l'invio — segnaposto che non lascia mai il browser.

Le due strade convivono: senza `PROXY_URL` tutto funziona come prima, e per i fornitori non coperti dal proxy Spectra continua a usare la chiave locale.

2.6 Ricominciare da capo

Tutto ciò che Spectra ricorda vive in `localStorage`: chat, cronologia, chiavi API, memoria persistente, schede di ripasso. Un pulsante "cancella cronologia" che togliesse solo le conversazioni non farebbe ripartire da capo — alla riapertura si ritroverebbe lo stesso stato.

Il punto delicato è che le chiavi hanno **due formati**: la mappa attuale (`bsi_api_keys`, una chiave per fornitore) e il formato singolo delle versioni precedenti (`bsi_api_key`). `getKeysMap()` migra automaticamente il secondo nel primo, quindi cancellare solo la mappa **fa riapparire la chiave vecchia** al primo accesso successivo. Vanno tolti entrambi — e con loro le cache dei modelli risolti, che senza chiave non hanno più significato.

Per questo l'elenco di ciò che l'applicazione scrive sta in un punto solo, `DATI_CANCELLABILI`, diviso in cinque gruppi con etichetta e descrizione. Il pannello di conferma si costruisce da lì: mostra quante voci esistono davvero per ciascun gruppo, disattiva quelli già vuoti e richiede un secondo clic esplicito. Spuntati di partenza solo chat e chiavi; memoria, ripasso e progressi del resto dell'app restano da scegliere a mano, perché cancellano lavoro che non c'entra con il ripartire da capo della conversazione.

Restano fuori di proposito la licenza PRO, il periodo di prova e l'identità del dispositivo: sono dati che l'utente non si aspetta di perdere svuotando una chat, e una licenza non si cancella per sbaglio. Il pannello lo dichiara.

Dopo la cancellazione viene azzerato anche lo stato in memoria del processo — elenco dei fornitori coperti dal proxy, modello risolto per ciascun provider — altrimenti resterebbe valido fino al ricaricamento della pagina, e Spectra continuerebbe a usare un modello scelto con una chiave che non esiste più.

2.7 Far reggere un carico vero alle chiavi gratuite

I piani gratuiti hanno due limiti diversi, e vanno affrontati in modo diverso.

Il tetto per richiesta. GitHub Models accetta 8.000 token in ingresso. Il costo fisso di Spectra è ~8.150 — 2.009 di prompt di sistema più **6.128 di sole definizioni degli strumenti** — quindi quel servizio non partirebbe nemmeno: fallirebbe prima che l'utente scriva una parola. La misura è stata fatta prima di scrivere il rimedio, e ha cambiato la soluzione.

Il rimedio non è tagliare la cronologia — è tagliare **il costo fisso**. Gli strumenti si pagano ad ogni turno, la conversazione è il contenuto: fra i due, è il costo fisso a doversi stringere per primo. `adattaAlBudget()` seleziona i **strumenti pertinenti** alla domanda invece di tutti e 32,

poi accorcia la cronologia con quel che resta.

La pertinenza si misura su una mappa di parole chiave tenuta **fuori** dal registro degli strumenti: le parole con cui la gente *chiede* una cosa non sono quelle con cui lo strumento è *documentato*. La descrizione di `astrofisica` parla di Wien e Stefan-Boltzmann; l'utente scrive «che temperatura ha questa nebulosa». Confrontando la domanda con la sola descrizione, su una domanda di astrochimica veniva scelto `farmacocinetica` — misurato, non ipotizzato. Il confronto è per radice (primi cinque caratteri) perché l'italiano flette: «nebulosa» e «nebulose» devono coincidere.

I limiti di frequenza. Un `429` sui piani gratuiti è quasi sempre il limite *al minuto*, non la quota esaurita: aspettare pochi secondi lo risolve. Il fornitore lo dice in `Retry-After`, oppure nel testo dell'errore («*Please try again in 7.5s*»); quando non lo dice si usa una crescita esponenziale con un po' di casualità, per non far ripartire tutte le schede aperte nello stesso istante. Se l'attesa richiesta è assurda non si aspetta affatto: meglio dire «quota finita» che lasciare l'utente davanti a una clessidra per un'ora.

La riserva fra fornitori. Quando una quota è davvero esaurita, il turno viene rifatto su un altro servizio per cui l'utente ha una chiave. È il motivo per cui più chiavi gratuite messe insieme reggono un carico che nessuna reggerebbe da sola.

Due vincoli non negoziabili:

- Si riparte dai messaggi **originali**, non dalla cronologia a metà. I turni

con chiamate a strumenti sono salvati nel formato nativo del fornitore precedente e non si possono passare a un altro. Si perde il lavoro del turno, si guadagna una risposta corretta.

- La riserva usa **solo servizi gratuiti**. Passare da solo a uno a pagamento

spenderebbe i soldi dell'utente senza che li abbia stanziati; se il servizio scelto era a pagamento resta comunque il primo tentativo.

E si passa oltre **soltanto** per quota esaurita. Un `401` o una richiesta malformata si ripeterebbero identici su ogni fornitore: provarli tutti per poi riportare l'ultimo errore a caso nasconderebbe la causa vera.

2.8 Modalità Nucleo, e dire cosa sta succedendo

La modalità. Un interruttore che cambia quattro cose insieme, perché nessuna delle quattro da sola sposterebbe l'ago: passa alla configurazione più capace fra quelle per cui esiste una chiave, porta i giri del ciclo agentico da 10 a 16 (una catena lunga di strumenti si esauriva a metà), chiede la profondità di ragionamento massima ai modelli che la espongono, e aggiunge al prompt l'istruzione di lavorare per passaggi verificati invece che per risposta rapida.

Il cambio di modello viene **dichiarato in chat**, e se il nuovo è a pagamento lo dice: passare di nascosto a un servizio a consumo significherebbe spendere i soldi dell'utente senza dirglielo. Per lo stesso motivo è un interruttore e non il comportamento predefinito — costa più quota, e su un servizio a pagamento costa denaro.

Dire cosa sta succedendo. Fra l'invio e la prima parola della risposta possono passare parecchi secondi: il modello ragiona, invoca strumenti, aspetta la rete. Prima l'utente vedeva una pausa muta e concludeva che si fosse bloccata.

L'intestazione ha ora un nucleo atomico i cui orbitali cambiano velocità e colore con lo stato — lento e ciano a riposo, rapido quando ragiona, ambra quando sta usando uno strumento — accompagnato da una riga che dice la stessa cosa a parole, per chi non guarda l'animazione. Gli stati si agganciano ai callback che il ciclo agentico già emetteva: non è stato aggiunto nessun meccanismo, solo reso visibile quello che c'era.

Tre difetti di impaginazione corretti nello stesso passaggio, tutti trovati misurando invece che guardando:

- Il campo di scrittura divideva la riga con quattro pulsanti e restava largo

156 px: il testo del segnaposto andava a capo e veniva **tagliato**. Ora il testo ha una riga propria e i comandi stanno sotto. La lunghezza del segnaposto è stata scelta provandola a 360 px di larghezza, non a occhio.

- Il titolo «Spectra — il copilota AI di BioSpecInfo» non entrava e si

troncava a metà. Resta il nome; il resto è diventato la riga di stato, che ora ha un uso.

- Gli orbitali del nucleo, ruotando, uscivano dal riquadro e finivano sopra il

nome.

2.9 Le capacità del fornitore, usate davvero

Tre cose che l'API di Anthropic offre e che non erano sfruttate. Verificate contro il riferimento ufficiale, non a memoria — due delle tre hanno un vincolo che a memoria non si sarebbe indovinato.

La cache del prompt. Il costo fisso di ogni richiesta è di circa 8.150 token: 2.000 di prompt di sistema più 6.128 di definizioni degli strumenti, identici ad ogni turno. Marcandoli si pagano una volta e poi si rileggono a una frazione del prezzo.

Il vincolo è che la cache funziona per *prefisso*: un solo byte diverso all'inizio la invalida tutta. Il prompt di Spectra però contiene anche la memoria dell'utente e il contesto della domanda, che cambiano ad ogni messaggio. Concatenandoli in un'unica stringa la cache **non avrebbe mai preso**. Ora il prompt viaggia in due blocchi — stabile e marcato, volatile e no — e il marcatore va anche sull'ultimo strumento, perché l'ordine di composizione è `tools` → `system` → `messages`.

Il sandbox Python, e perché esclude la ricerca web. `code_execution` dà al modello un ambiente con sympy, numpy, scipy e matplotlib: integrazione numerica, algebra simbolica, diagonalizzazione di matrici — cose che i 32 risolutori non coprono perché non si possono prevedere tutte.

Ma la ricerca web di nuova generazione **porta già dentro un ambiente di esecuzione** (filtra i risultati scrivendo codice), e dichiarare anche `code_execution` ne creerebbe un secondo, confondendo il modello. Sono alternative, non complementari. La scelta segue la

Modalità Nucleo: normalmente ricerca web e lettura di pagine, in Nucleo il sandbox — perché quando si chiede il massimo della potenza il problema è di calcolo, non di documentazione. Un test verifica che i due non compaiano mai insieme.

La lettura di pagine. `web_fetch` affianca la ricerca: se l'utente incolla il link di un articolo, Spectra lo legge invece di limitarsi a cercarne il titolo.

I risultati del sandbox arrivano come `bash_code_execution_tool_result` e `text_editor_code_execution_tool_result` — non come un generico `code_execution_tool_result` — e vanno conservati nel turno dell'assistente come i blocchi della ricerca web, altrimenti un turno messo in pausa dal server non si può riprendere.

---

3. I 35 strumenti

L'elenco è quello **misurato** in `window.BSI_AI_TOOLS` sull'applicazione in esecuzione, non trascritto a memoria: questo documento ne dichiarava 32 quando erano già 35, e tre — `analizza_molecola`, `disegna_molecola`, `mostra_spettri` — non comparivano affatto.

Area	Strumenti
Calcolo generale	`calcola`, `risolvi_equazione`, `analisi_dati`
Chimica generale	`bilancia_equazione`, `stechiometria`, `massa_molecolare`, `converti_unita`, `costante_fisica`
Chimica fisica	`termodinamica`, `equilibrio_acido_base`, `cinetica`, `gas_e_soluzioni`, `elettrochimica`
Struttura e spettri	`spettroscopia`, `quantistica_e_spettroscopia`, `cristallografia`, `analizza_molecola`, `disegna_molecola`, `mostra_spettri`
Scienze della vita	`biochimica`, `farmacocinetica`, `valuta_druglikeness`
Fisica	`astrofisica`, `nucleare`, `statistica_inferenziale`
Banche dati esterne	`cerca_pubchem` (NIH), `cerca_letteratura` (PubMed)
Dati interni	`cerca_nel_database` (9 dataset), `cerca_molecola`
Controllo app	`naviga_sezione` (95 sezioni), `apri_strumento` (12 laboratori), `stato_app`
Memoria	`ricorda`, `ricordi`
Animazioni	`apri_animazione` (6 meccanismi di reazione)

> **La ricerca web non è fra questi trentacinque**, ed è bene distinguerla. I trentacinque sono eseguiti **dall'applicazione**, nel browser dell'utente. Sui fornitori che li offrono, l'agente aggiunge allo schema due strumenti eseguiti **dal fornitore** — `web_search` e `web_fetch`, con un tetto di usi per turno. Contarli insieme agli altri li farebbe sembrare disponibili sempre, mentre dipendono dal fornitore scelto.

3.1 Dataset interni esposti

297 reazioni di sintesi · 118 elementi · 67 amminoacidi · 263 farmaci · 63 patologie · 46 strategie retrosintetiche · 36 interazioni farmacologiche · 29 vie metaboliche · 29 potenziali redox.

I farmaci e le strategie sono i due numeri che `tools/verifica-affermazioni.js` misura sull'applicazione in esecuzione a ogni batteria: qui erano rimasti a 143 e 39 mentre l'app ne contava 178 e 46.

Sono i dati curati dall'autore per l'applicazione: l'agente li consulta come fonte primaria e ha istruzione di **segnalare le discrepanze** fra database e propria memoria invece di appianarle silenziosamente.

---

4. Decisioni ingegneristiche rilevanti

Questa sezione documenta le scelte non ovvie e il motivo per cui sono state prese. È la parte del documento che descrive il ragionamento progettuale.

4.1 Motore di calcolo senza `eval()`

Problema. Servire calcoli arbitrari di chimica fisica richiede un valutatore di espressioni. `eval()` lo risolverebbe in poche righe.

Perché è stato escluso. L'agente legge contenuti esterni non fidati (risultati PubChem, abstract PubMed, pagine web). Un'iniezione in quei contenuti potrebbe indurre il modello a emettere un'espressione ostile; con `eval()` quella stringa verrebbe eseguita nel contesto della pagina, **dove risiede la chiave API dell'utente**.

Soluzione. Parser a discesa ricorsiva con insieme *chiuso* di 27 funzioni e costanti. Non ha accesso all'ambito globale: può solo fare aritmetica. Verificato con 10 tentativi di evasione (`window.localStorage`, `constructor`, `this`, `fetch(...)`, chiamate a funzioni non in whitelist), tutti respinti, anche eseguendo il test in un browser reale.

4.2 Conservazione dei blocchi di ragionamento

Sui modelli con ragionamento adattivo i blocchi `thinking` fanno parte del turno assistente e devono essere **rimandati indietro identici, firma crittografica inclusa**, nel giro successivo di *tool use*. Scartarli fa rifiutare la richiesta.

L'implementazione accumula `thinking_delta` e `signature_delta` dallo stream e li reinserisce **in testa** al turno ricostruito. Verificato in test d'integrazione: firma preservata, ordine corretto.

4.3 Ripresa dei turni sospesi (`pause_turn`)

La ricerca web è uno strumento eseguito sui server del fornitore. Quando il suo ciclo interno esaurisce le iterazioni disponibili, la risposta termina con `stop_reason: "pause_turn"`.

Il turno va ripreso **rimandando indietro il turno assistente così com'è, senza aggiungere alcun messaggio utente**: è il server a riconoscere il blocco `server_tool_use` in coda e a riprendere da lì. Aggiungere un "continua" spezzerebbe il meccanismo.

4.4 Configurazione per modello, non globale

I quattro livelli Claude non accettano gli stessi parametri:

Modello	`effort`	Ricerca web	Note
Fable 5.1	`xhigh`	sì (8)	fallback su rifiuto; ragionamento sempre attivo
Opus 5	`high`	sì (6)	predefinito
Sonnet 5	`high`	sì (5)	
Haiku 4.5	**non supportato**	**non supportata**	inviarli produce HTTP 400

Su tutti i modelli recenti `temperature`, `top_p` e `budget_tokens` sono stati rimossi dall'API e la loro presenza causa un errore: nessuno dei quattro li invia. Ogni parametro è quindi condizionato al fornitore, non impostato globalmente.

4.5 Ingresso multimodale e lettura dei materiali

L'agente accetta immagini, PDF e file di testo, anche molti insieme, trattati come pagine consecutive di un unico documento. Tre vincoli hanno guidato l'implementazione:

- **Le immagini vengono ridimensionate a 2000 px** e ricompresse in JPEG prima

dell'invio. Una foto da telefono passa da circa 1 MB a 440 KB: senza questo passaggio una singola immagine avvicinerebbe il tetto di richiesta e moltiplicherebbe il costo in token. 2000 px sono il minimo per leggere una grafia minuta — a 1600 px la scrittura piccola diventava illeggibile.

- **Il tetto di pagine dei PDF dipende dal modello**, non è una costante: 600

pagine per i modelli con finestra da 1M, 100 per quelli da 200K. Il conteggio avviene leggendo gli oggetti `/Type /Page` nei byte del file, senza librerie.

- **Nella cronologia va una miniatura da 160 px**, non l'immagine inviata.

Salvare quella grande (440 KB l'una) riempiva `localStorage` in una decina di foto, e a quota piena ogni scrittura successiva fallisce in silenzio.

Sei azioni rapide trasformano il materiale in un prodotto di studio (riassunto, mappa concettuale, schema, flashcard, domande d'esame, trascrizione) e due lo traducono. Per le trascrizioni l'agente ha istruzione di scrivere `[illeggibile]` invece di indovinare: un termine chimico inventato è peggio di una lacuna dichiarata.

4.6 Renderer di grafi proprio, senza CDN

Le mappe concettuali sono prodotte dal modello come diagrammi Mermaid e disegnate da un renderer scritto nell'applicazione (~190 righe): livelli per cammino più lungo dalle radici con interruzione dei cicli, riordino per baricentro dei predecessori su tre passate per ridurre gli incroci, output SVG con archi di Bézier.

La scelta di non caricare Mermaid da CDN nasce dalla natura offline-first dell'app: una mappa deve potersi disegnare anche senza rete. Le sintassi non coperte vengono riconosciute e lasciate come blocco di codice leggibile, invece di essere disegnate male.

4.7 Comando vocale con parola di attivazione

Ascolto continuo attivabile dall'utente: la frase «Hey Spectra» seguita dal comando lo invia automaticamente. Due dettagli non ovvi:

- il riconoscimento vocale del browser **termina da solo** dopo qualche secondo

di silenzio e va riavviato, altrimenti l'ascolto muore alla prima pausa;

- vengono esaminate **tutte le alternative di trascrizione**, non solo la

prima, e sono accettate le varianti fonetiche più comuni: il riconoscimento italiano rende "Spectra" in molti modi diversi.

Il rifiuto del permesso al microfono interrompe il ciclo invece di ritentare all'infinito. Il termine "spettroscopia", frequentissimo in questo dominio, è stato verificato come non attivante.

4.8 Guardia sui risultati non finiti

Un audit sistematico ha rivelato che, con input degeneri legittimi (lunghezza d'onda nulla, volume nullo, emivita nulla, transizione fra livelli identici), otto risolutori restituivano `ok: true` con `Infinity` o `NaN` fra i campi.

È il guasto più insidioso in questo contesto: un'eccezione si nota, un valore formalmente valido ma privo di senso viene riportato all'utente come corretto. La correzione è un controllo unico nel punto in cui transitano tutti i risultati: se un campo numerico non è finito, il risultato diventa un errore esplicito che nomina i campi e indica la causa probabile. Vale anche per gli strumenti aggiunti in futuro.

4.9 Degradazione controllata

- **Modelli senza *function calling* affidabile** (alcuni gratuiti): al primo

errore riconducibile agli strumenti, la richiesta viene ripetuta una volta in modalità solo testo, invece di fallire.

- **Timeout di inattività (45 s)**, azzerato ad ogni byte ricevuto: una risposta

lunga non viene troncata, ma una connessione morta viene segnalata subito con un messaggio esplicito.

- **Quota di `localStorage`**: lo storico è limitato (30 conversazioni, 100

messaggi) con potatura progressiva. Senza questo limite il superamento della quota faceva fallire *in silenzio* ogni scrittura successiva, **compreso il salvataggio della chiave API**.

- **Rifiuto dei classificatori** (`stop_reason: "refusal"`, HTTP 200): rilevato

e comunicato, invece di presentarsi come risposta vuota.

---

5. Verifica

La verifica è stata condotta su tre livelli, con esiti registrati.

5.1 Correttezza numerica — confronto con valori di riferimento

Ambito	Caso	Atteso	Ottenuto
Bilanciamento	`KMnO₄ + HCl → …`	2:16:2:2:8:5	✓ esatto
Massa molecolare	`K₄[Fe(CN)₆]` (annidata)	368,35	368,345
Massa molecolare	`CuSO₄·5H₂O` (idrato)	249,68	249,677
Equilibrio	acido acetico 0,1 M, K_a $1{,}8\cdot10^{-5}$	pH 2,874	2,875
Termodinamica	ΔG (N_2O_4/NO_2) a 298 K	+4,79 kJ/mol	✓
Cinetica	E_a da k(300 K) e k(320 K)	55,3 kJ/mol	55,33
Gas reali	van der Waals CO_2	scostamento −10 %	−10,13 %
Spettroscopia	M+2 del bromo	97,3 %	✓
Biochimica	glicilglicina	132,12 Da	✓
Farmacocinetica	t½ da V_d 50 L, Cl 5 L/h	6,93 h	✓
Astrofisica	picco di emissione solare (Wien)	501,5 nm	501,52
Astrofisica	velocità di fuga terrestre	11,19 km/s	11,186
Nucleare	energia di legame ^{56}Fe	8,79 MeV/nucleone	8,790
Statistica	t critico, 95 %, df = 4	2,7764	✓
Cristallografia	densità del rame (fcc)	8,96 g/cm³	8,935

I p-value non sono tabulati: sono calcolati con la funzione beta incompleta regolarizzata (frazione continua di Lentz) e la gamma incompleta.

5.2 Robustezza — casi che devono fallire

Verificato che vengano **respinti**: equazioni non bilanciabili, formule con parentesi non bilanciate o elementi inesistenti, specie assenti dall'equazione, tipi di database non validi, e i 10 tentativi di iniezione nel motore di calcolo.

5.3 Integrazione — browser reale

Test automatizzati con Chromium *headless* su tutte le 14 pagine dell'applicazione: nessun errore JavaScript, nessuna risorsa mancante. Percorse una per una le 95 sezioni di `index.html` e i 18 tab della sezione Astrochimica. Ispezionato il corpo delle richieste per i quattro livelli Claude, e simulato un ciclo completo con ricerca web, sospensione e ripresa del turno.

6. Riepilogo quantitativo

Metrica	Valore
Righe del componente	6.254
Strumenti	32
Configurazioni di modello	10 (3 gratuite)
Aree scientifiche coperte	13
Record nei dataset interni esposti	oltre 800
Giri massimi del ciclo agentico	10
Cronologia inviata al modello	40 turni
Dipendenze runtime	nessuna

---

7. Limiti dichiarati

Documentati per onestà tecnica; sono scelte consapevoli, non omissioni.

- **La chiave API risiede nel browser** — a meno di pubblicare il proxy.

Nella configurazione predefinita la chiave non lascia il dispositivo, ma è accessibile a chi ha accesso a quel browser, e ogni utente deve procurarsi la propria. Il Worker della sezione 2.5 toglie questo limite spostando le chiavi su un server, al prezzo di un componente da mantenere fuori dalla pagina: è una scelta di distribuzione, non un requisito.

- **Gli strumenti di rete dipendono dalla disponibilità dei servizi.** PubChem

e PubMed sono interrogati direttamente dal browser; in caso di irraggiungibilità lo strumento restituisce un errore esplicito e l'agente ha istruzione di dichiararlo, non di sostituirlo con una stima.

- **I risolutori applicano modelli semplificati** dove la letteratura ne

prevede di più raffinati (per esempio Woodward–Fieser per l'UV, o le rese in ATP con rapporti P/O medi). Ogni strumento dichiara il metodo usato nel proprio risultato.

- **La verifica è funzionale e numerica**, non formale: non esiste prova di

correttezza dei risolutori, ma un insieme di casi di riferimento tratti dalla letteratura didattica.

Documento redatto a corredo del dossier tecnico BioSpecInfo. Vedi anche: `01-Software-Architecture-Document.md`, `02-Verification-Validation-Report.md`, `03-Security-Privacy-Compliance.md`.